❶ The filters encapsulate the majority of logic common to multiple action methods.

❷ Placing filters at the action level limits them to a single action.

❸ The intent of the action, return a Recipe view model, is much clearer.

❹ Placing filters at the action level can control the order in which they execute.

❺ The intent of the action, update a Recipe, is much clearer.

I think you'll have to agree that the controller in listing 22.2 is much easier to read! In this section you'll refactor the controller bit by bit, removing cross-cutting code to get to something more manageable. All the filters we'll create in this section will use the sync filter interfaces. I'll leave it to you, as an exercise, to create their async counterparts. We'll start by looking at authorization filters and how they relate to security in ASP.NET Core.

22.1.1 Authorization filters: Protecting your APIs

Authentication and *authorization* are related, fundamental concepts in security that we'll be looking at in detail in chapters 23 and 24.

DEFINITION *Authentication* is concerned with determining who made a request. *Authorization* is concerned with what a user is allowed to access.

Authorization filters run first in the MVC filter pipeline, before any other filters. They control access to the action method by immediately short-circuiting the pipeline when a request doesn't meet the necessary requirements.

ASP.NET Core has a built-in authorization framework that you should use when you need to protect your MVC application or your web APIs. You can configure this framework with custom policies that let you finely control access to your actions.